PROJECT OVERVIEW

MailShield

Explainable Spam Detection. Transparent Decisions. Privacy by Design.

Explainable ML | Multilingual | Phishing Risk Engine | Offline Desktop | Evaluation Lab

View Repository | Report a Bug | Browse Releases

MailShield is an explainable machine-learning platform for detecting spam, identifying phishing risks, and understanding why an email was classified a certain way.

It combines multilingual text classification, feature-level explanations, an independent phishing-risk engine, and an interactive evaluation environment in one application—with a standalone desktop experience designed for local, offline analysis.

> **No black-box verdicts. No mandatory cloud processing for local analysis. Just explainable email security.**

6 languages supported	**3** classifier workflows	**4** Gmail scan batch sizes	**0** send / modify / delete permissions

01 What makes MailShield different?

Most spam classifiers return a label and a probability. MailShield goes further: it helps users inspect the evidence behind a decision, evaluate the model, and examine phishing indicators separately from the classifier's prediction.

Explainable ML

Inspect the feature contributions behind classification decisions.

Multilingual Detection

Supports English, Hindi, Tamil, Telugu, Kannada, and Hinglish.

Independent Phishing Analysis

Evaluate suspicious URLs, domain patterns, credential-harvesting paths, and urgency indicators separately from the ML score.

Offline Desktop Mode

Analyse email text locally without requiring a hosted backend for the core classification workflow.

Gmail Integration

Scan inbox messages using Google OAuth with read-only access.

Evaluation Lab

Examine confusion matrices, false positives, false negatives, language-specific performance, and validation-threshold trade-offs.

Privacy-Conscious Design

Avoid retaining email bodies by default, subject to the application's configured workflow and explicit user choices.

02 Two ways to use MailShield

Standalone Desktop
FOR EVERYDAY USE

The desktop application bundles its backend executable so users do not need to install Python or create a Python virtual environment on the target Mac.

Core local analysis is designed to work offline after installation, provided the required application assets are bundled. Gmail inbox scanning requires an internet connection and Google authentication.

Source Code
FOR DEVELOPERS

Prefer to inspect the implementation, run tests, train models, or modify MailShield? Clone the repository or download the source ZIP and follow the development setup below.

Source code is not the same as a prebuilt desktop application: running from source requires the relevant development tools and dependencies.

Platform availability: The documented standalone release targets macOS on Apple Silicon (`darwin-arm64`). Windows, Intel Mac, and Linux desktop packages are not included in this release.

03 Key Features

Gmail Inbox Scanner

Connect a Gmail account to analyse selected inbox messages without granting permission to send, modify, or delete emails.

- Google OAuth 2.0 with PKCE for supported authentication flows.
- Read-only Gmail access using `gmail.readonly` and `userinfo.email`.
- Batch scanning for 5, 10, 25, or 50 messages.
- Classification breakdowns and explanations for scanned messages.
- Encrypted token storage using AES-256-GCM with key derivation and identity binding.

Privacy note: Gmail scanning requires an internet connection. Token encryption and email-retention behaviour depend on the implemented storage and configuration; review the privacy documentation before connecting a personal account.

Multilingual Spam Detection

MailShield's text-classification pipeline supports six target language categories:

English	Hindi	Tamil	Telugu	Kannada	Hinglish Romanized Indic
en	hi	ta	te	kn	hinglish

The model combines word-level n-grams with character-level features to help recognise spelling variations, transliteration, and modified spam expressions.

Explainable Machine Learning

MailShield is built to make classification decisions easier to inspect.

- Feature-level contribution analysis.
- Support for Logistic Regression, Multinomial Naive Bayes, and Linear SVM workflows.
- Calibrated probabilities where supported by the model pipeline.
- A distinction between probability estimates and raw decision-margin scores.

A prediction is not presented as unquestionable truth. The supporting evidence matters, too.

Independent Phishing Risk Engine

The phishing-risk layer evaluates suspicious patterns independently of the ML classifier's probability. Indicators include:

- IP-address-based hostnames.
- URL shorteners.
- Suspicious banking-domain patterns.
- Credential-related paths such as `/login` and `/kyc`.
- Urgency and credential-harvesting language.

This separation helps prevent the phishing heuristic from being confused with the classifier's own prediction.

Important: These indicators are heuristics, not proof that a message is malicious or legitimate.

Interactive Evaluation Lab

Evaluate the model instead of relying on a single accuracy figure.

- Model selection and validation inspection.
- Per-language filtering.
- Confusion matrices.
- False-positive and false-negative investigation.
- Dataset provenance inspection.
- Interactive classification experiments.
- Threshold simulation for examining precision/recall trade-offs.

Email and File Analysis

Analyse email content by pasting text or, where supported, uploading `.eml` and `.txt` files. The interface includes sample messages for exploring spam, phishing, financial scams, ordinary business emails, travel receipts, and multilingual banking alerts.

Model Lab and Threshold Simulation

Explore how changing the classification threshold affects model behaviour. The threshold simulator uses validation predictions to illustrate trade-offs involving false-positive rate, recall, and confusion matrices. Results depend on the selected model, dataset, and evaluation configuration.

History and Audit Export

The application includes analysis-history tools for searching, filtering, sorting, deleting records, and exporting supported results to CSV or JSON. Export functionality includes protections against formula injection.

Privacy by Design

MailShield is designed to minimise unnecessary email-content retention.

- Local desktop analysis for supported workflows.
- No mandatory hosted classification service for core offline analysis.
- Email-body retention disabled by default in the documented workflow.
- Read-only Gmail permissions.
- Protected token storage for supported OAuth flows.

Privacy depends on the actual execution path and configuration. Connecting Gmail, enabling persistence, exporting results, or using an online deployment can change the data flows involved.

04 Architecture

MailShield separates the user interface, classification pipeline, phishing-risk analysis, and evaluation tooling.



The architecture supports both a standalone desktop application and a web/API deployment. A local desktop backend is not the same as a publicly hosted API.

05 Technology Stack

Frontend	React · TypeScript · Vite · Tailwind CSS
Backend	Python · FastAPI · Pydantic · SQLAlchemy
Machine Learning	scikit-learn · pandas · NumPy · SciPy
Classification	TF-IDF features · Logistic Regression · Multinomial Naive Bayes · Linear SVM
Desktop	Electron · PyInstaller · bundled Python backend
Testing	Pytest · Vitest · React Testing Library
Storage	SQLite for supported local desktop workflows

Exact dependency versions are maintained in the project's dependency and configuration files.

06 Installation

Option A — Standalone desktop release

RECOMMENDED FOR USERS WHO SIMPLY WANT TO RUN MAILSHIELD

1. Open MailShield Releases.
2. Download the published macOS Apple Silicon archive, if available.
3. Extract the archive.
4. Move **MailShield.app** into Applications.
5. Open the application and start analysing supported email content.

No separate Python installation, `pip install`, or virtual environment should be required for a correctly packaged standalone build.

Option B — Download the source ZIP

FOR DEVELOPERS

1. Open the repository.
2. Select **Code → Download ZIP**.
3. Extract the archive.
4. Open a terminal inside the extracted project folder.
5. Follow the developer setup below.

Downloading source code does not automatically install the dependencies or create a ready-to-run desktop application.

Release availability: A downloadable archive must first be published under GitHub Releases. If no release asset is listed, use Option B or wait for the packaged release to be published.

07 Developer Setup

PREREQUISITES

- Python 3.11 or newer, using a version supported by the project's dependencies.
- Node.js 20 or newer and npm.
- Git, if cloning the repository.

terminal

```
# 1. Clone the repository
git clone https://github.com/priya-kumaricodes/MailSheild.git
cd MailSheild

# 2. Create a Python environment (macOS / Linux)
python3 -m venv .venv
source .venv/bin/activate

# 3. Install project dependencies
npm --prefix apps/web install
npm --prefix apps/desktop install

# 4. Build the frontend
npm --prefix apps/web run build

# 5. Build the desktop application
npm --prefix apps/desktop run build
npm --prefix apps/desktop run package
```

The packaging command should produce the desktop distributable configured for the current platform. Confirm the output path and supported architecture in the package configuration before distributing it.

08 Testing and Quality Checks

Run the checks available in the current checkout:

tests

```
.venv/bin/pytest -v
npm --prefix apps/web test
npm --prefix apps/web run build
npm --prefix apps/desktop run build

# standalone packaging verification
npm --prefix apps/desktop run package
```

Test counts and packaging results can change between releases. Consult the actual test output and release report before claiming a particular number of passing tests.

> **Dependencies:** Install the Python and JavaScript dependencies specified by the repository's current configuration. If the project provides dedicated dependency files, use those rather than maintaining a separate, manually assembled package list. Install backend and ML dependencies according to the repository's dependency files and documented setup.

For backend development, API testing, or web deployment, follow the project's technical documentation. Those are developer workflows and are separate from running the packaged offline desktop application.

09 Documentation

The repository's `docs/` directory contains the detailed product and engineering specifications, where available:

Product Definition Specification `docs/PDS.md`	**Technical Design Specification** `docs/TDS.md`
Architecture `docs/ARCHITECTURE.md`	**API Documentation** `docs/API.md`
Data Dictionary `docs/DATA_DICTIONARY.md`	**Model Card** `docs/MODEL_CARD.md`
Dataset Card `docs/DATASET_CARD.md`	**Privacy Documentation** `docs/PRIVACY.md`
Threat Model `docs/THREAT_MODEL.md`	**Test Plan** `docs/TEST_PLAN.md`
User Guide `docs/USER_GUIDE.md`	

These links refer to files in the repository; the files should be present in the published branch for the links to work.

10 Limitations and Security Notes

- The documented standalone release targets macOS Apple Silicon only.
- Gmail inbox integration requires internet access and Google authentication.
- Local offline classification and online Gmail scanning are distinct workflows.
- Classification scores are estimates, not guarantees.
- Phishing indicators are heuristics and may produce false positives or false negatives.
- Evaluation results depend on the dataset and validation methodology.
- Never commit real `.env` files, API keys, OAuth client secrets, access tokens, private emails, or personal datasets.
- Before connecting a Gmail account, review the OAuth scopes, token-storage implementation, and privacy documentation.

11 About the Developer

PK Priya Kumari
DEVELOPER

Creator and Developer of MailShield

GitHub @priya-kumaricodes

LinkedIn Priya Kumari

AT Anurag Tiwari
CONTRIBUTIONS

Contributor; provided limited assistance during development.

GitHub @anurag-tiwaricodes

LinkedIn Anurag Tiwari

12 License

License

MailShield is intended to be distributed under the MIT License. See the repository's `LICENSE` file for the applicable terms.

Dataset and sample-email licensing may differ from the software license. Review the relevant dataset documentation before redistributing sample data.

MailShield — Security you can inspect. Decisions you can explain.